

ZroAct 통합 파이프라인 구현 현황 보고서

1. 개요

ZroAct는 산업 현장 CCTV 영상에서 작업자의 행동을 분석하고, 제한 구역 침입과 같은 위험 상황을 탐지하기 위한 시스템이다.

현재 구현한 통합 파이프라인은 기존의 Stage 1 행동 탐지 모델과 Stage 2 VLM 위험 판단 구조를 연결하여, 입력 영상 또는 프레임 폴더를 전달하면 최종 위험 로그와 오버레이 이미지까지 자동으로 생성하는 오프라인 실행 프레임워크이다.

현재 구조는 실시간 CCTV 서버가 아니라, 실제 파이프라인의 처리 과정을 하나의 명령으로 재현할 수 있도록 만든 통합 실행 환경이다.

2. 전체 처리 구조

기존 실험 구조는 이미 생성된 Stage 1 결과를 이용하여 Stage 2만 실행하는 방식이었다.

```
기존 Stage 1 결과
→ Stage 2 request manifest 생성
→ Qwen3.5 VLM 추론
→ summary.csv 저장
```

현재 통합 파이프라인은 여기에 Stage 1 실행과 최종 로그 생성 과정을 추가하였다.

```
입력 영상 또는 프레임 폴더
→ Stage 1 YOW0v3 행동 탐지
→ Stage 2 request manifest 생성
→ Stage 2 Qwen3.5 VLM 위험 판단
→ 전체 상태 로그 생성
→ 위험 이벤트 로그 생성
→ 위험 상태 overlay 이미지 생성
→ 단계별 실행 시간 저장
```

3. 프로젝트 경로 구성

현재 Stage 1과 Stage 2는 서로 다른 프로젝트 폴더와 Conda 환경에서 실행된다.

```
/home/capstone2/
├── zroact-stage1/
│   └── YOW0v3/
```

```
├─ zroact-stage2/  
│   └─ benchmark2/  
│       └─ pipeline/  
├─ labeling/  
└─ miniconda3/
```

통합 실행 파일은 다음 위치에 있다.

```
/home/capstone2/zroact-stage2/pipeline/main.py
```

이 파일은 Stage 1과 Stage 2 모델을 새로 구현하지 않는다. 기존 실행 파일을 subprocess 방식으로 호출하여 두 단계를 연결한다.

Stage 1과 Stage 2는 의존성이 다를 수 있으므로 Conda 환경을 하나로 합치지 않고 분리하여 실행한다.

```
Stage 1: conda run -n yowov3 ...  
Stage 2: conda run -n qwen35 ...
```

4. Stage 1: YOWOv3 행동 탐지

4.1 역할

Stage 1은 YOWOv3를 이용하여 입력 영상 또는 프레임에서 작업자를 탐지하고, 프레임별 행동 정보를 생성한다.

주요 출력은 다음과 같다.

- 작업자 bounding box
- 작업자별 action label
- bbox가 표시된 이미지
- 프레임별 bbox 및 action label JSON

4.2 실행 위치

```
/home/capstone2/zroact-stage1/YOWOv3
```

4.3 실행 방식

```
conda run -n yowov3 python main.py -m custom_frame_infer ...
```

4.4 Stage 1 출력 예시

```
benchmark2/runs/{run_id}/stage1/  
└─ climb-over-fence_plant_rgb/  
  └─ images/  
    └─ intrusion_climb-over-fence_rgb_0007_cctv3/  
      └─ intrusion_climb-over-fence_rgb_0007_cctv3_t000016.jpg  
      └─ intrusion_climb-over-fence_rgb_0007_cctv3_t000026.jpg  
      └─ ...  
    └─ labels/  
      └─ intrusion_climb-over-fence_rgb_0007_cctv3.json
```

`images`에는 bbox가 그려진 이미지가 저장된다.

`labels`에는 각 프레임의 bbox 좌표와 action label이 JSON 형태로 저장된다.

5. Stage 1 Sample Rate

Stage 1은 처리할 프레임 간격을 조절할 수 있다.

```
--stage1-sample-rate 1  
→ 거의 모든 프레임을 처리  
  
--stage1-sample-rate 10  
→ 10프레임 간격으로 처리
```

현재 Stage 2 입력은 10프레임 간격의 이미지 3장을 사용하는 구조이다.

```
[t, t+10, t+20]
```

따라서 Stage 2 실험용 데이터 생성에서는 `sample_rate=10`이 효율적이다.

측정 결과

설정	Stage 1 처리 시간
<code>sample_rate=1</code>	약 65초
<code>sample_rate=10</code>	약 14초

단, `sample_rate=10`은 통합 실행 시간을 줄이기 위한 실험 설정이다. 실제 운영 구조에서 모든 프레임의 Stage 1 결과가 필요하다면 `sample_rate=1` 기반 처리 또는 스트리밍 구조를 별도로 검토해야 한다.

6. Stage 2 Request Manifest

6.1 Manifest의 의미

Manifest는 Stage 2 VLM에 전달할 입력 목록이다.

한 줄이 VLM 호출 1회에 해당하며, 다음 정보를 포함한다.

- request_id
- video_id
- 선택된 이미지 3장의 frame index
- 각 이미지의 시간 정보
- Stage 1 window를 고려한 sequence coverage
- 이미지 파일 경로
- 각 이미지에 대응하는 Stage 1 action label

Manifest는 VLM 추론 결과가 아니라, VLM에 넣을 입력을 미리 구성하는 전처리 결과이다.

6.2 샘플링 규칙

Stage 2 request는 10프레임 간격의 이미지 3장으로 구성한다.

```
Request 1: [16, 26, 36]
Request 2: [46, 56, 66]
Request 3: [76, 86, 96]
```

즉, 하나의 request 안에서는 10프레임 간격으로 이미지 3장을 선택하고, 다음 request는 30프레임 뒤에서 시작한다.

```
within-sequence gap = 10 frames
request stride = 30 frames
```

30 FPS 영상 기준으로 약 1초마다 VLM을 한 번 호출하는 구조이다.

6.3 Sequence Coverage

Stage 1은 `window=16`, `stride=1` 구조이다.

따라서 frame 16, 26, 36의 action label은 각각 이전 16프레임의 시계열 정보를 반영한다.

```
frame 16 action label → frames 1~16
frame 26 action label → frames 11~26
frame 36 action label → frames 21~36
```

결과적으로 Stage 2 request `[16, 26, 36]`은 단순히 이미지 3장만 의미하는 것이 아니라, 약 `frames 1~36`의 시간 흐름을 압축하여 전달하는 구조이다.

6.4 Manifest 예시

```
{
  "request_id": "intrusion_climb-over-fence_rgb_0007_cctv3_f000016_000026_000036",
  "video_id": "intrusion_climb-over-fence_rgb_0007_cctv3",
  "frame_indices": [16, 26, 36],
  "frame_times_sec": [0.533, 0.867, 1.2],
  "sequence_coverage": {
    "start_frame": 1,
    "end_frame": 36
  },
  "images": [
    "...t000016.jpg",
    "...t000026.jpg",
    "...t000036.jpg"
  ],
  "stage1_actions": [
    "stand, carry/hold (an object)",
    "stand, carry/hold (an object)",
    "stand, carry/hold (an object)"
  ]
}
```

7. Stage 2: Qwen3.5 VLM 위험 판단

7.1 역할

Stage 2는 이미지 3장과 Stage 1 action label을 함께 받아 제한 구역 침입 위험 상태를 판단한다.

7.2 실행 위치

```
/home/capstone2/zroact-stage2/benchmark2/scripts/run_stage2_vlm.py
```

7.3 실행 방식

```
conda run -n qwen35 python benchmark2/scripts/run_stage2_vlm.py ...
```

7.4 기본 모델

```
benchmark2/models/Qwen3.5-2B
```

7.5 VLM 입력 구조

- 이미지 3장
- 각 이미지의 frame index
- 각 이미지의 time sec
- 각 이미지의 Stage 1 action label
- 침입 탐지 전용 prompt

7.6 VLM 출력 구조

출력은 단순한 JSON 형식으로 유지한다.

```
{"risk_state": "normal"}
```

```
{"risk_state": "unsafe"}
```

```
{"risk_state": "danger"}
```

8. 침입 탐지 Prompt 기준

현재 Stage 2 Prompt는 낙상이나 충돌이 아니라 `climb-over-fence` 또는 제한 구역 침입 행동에 맞춰 구성한다.

판단 기준

상태	의미
<code>normal</code>	침입 관련 행동이 보이지 않음
<code>unsafe</code>	펜스 또는 경계 근처에서 접근, 확인, 접촉, 대기, 준비 행동이 보이지만 아직 등반은 시작되지 않음
<code>danger</code>	발판 또는 펜스 위에 발을 올리거나, 펜스를 잡고 체중을 지지하거나, 실제 등반 또는 경계 통과가 시작됨

Stage 1 action label은 주된 판단 근거가 아니라 시간 흐름을 보조하는 정보로 사용한다.

이미지 = 주요 증거
Stage 1 action label = 보조 정보

9. Human GT 라벨 적용 방향

현재 통합 실행 구조는 GT가 없어도 동작한다.

다만 추후 정량 평가와 학습을 위해서는 YOLO txt 기반 Human GT를 연결해야 한다.

9.1 Frame-level GT

각 txt 파일의 bbox 좌표는 사용하지 않고, `class_id` 와 프레임 번호만 사용한다.

```
0 = unsafe
1 = danger
2 = normal
```

한 프레임 안에 여러 class가 있을 경우 위험도가 높은 라벨을 우선한다.

```
danger > unsafe > normal
```

즉:

```
class_id 1이 하나라도 존재 → danger
그렇지 않고 class_id 0 존재 → unsafe
그 외 → normal
```

9.2 Sequence-level GT

기본 비교용 GT는 실제 sampled frame 3장을 기준으로 만들 수 있다.

```
3장 중 danger가 하나라도 존재 → danger
그렇지 않고 unsafe가 하나라도 존재 → unsafe
그 외 → normal
```

다만 최종 학습 및 평가 GT는 Stage 1 window coverage 전체를 반영하는 방향으로 확장해야 한다.

```
sampled-only GT
→ baseline 비교용

coverage-based GT
→ 최종 학습 및 정량 평가용
```

Coverage 기반 GT에서는 request의 `sequence_coverage` 안에 포함된 전체 프레임 GT 비율을 계산하고, 애매한 transition 구간은 `ignore` 로 처리하는 방식이 적절하다.

10. 최종 출력 구조

통합 실행 결과는 다음 경로에 저장된다.

```
benchmark2/runs/{run_id}/final/
```

파일 구조는 다음과 같다.

```
final/
├─ status_logs.jsonl
├─ status_logs.csv
├─ risk_logs.jsonl
├─ risk_logs.csv
├─ overlay_images/
├─ timings.json
└─ timings.csv
```

파일별 의미

파일	의미
status_logs	normal, unsafe, danger 전체 상태 기록
risk_logs	unsafe, danger 위험 이벤트만 기록
overlay_images	위험 이벤트 발생 시 risk badge를 추가한 이미지
timings	단계별 처리 시간 기록

11. 위험 로그 예시

```
{
  "event_id": "evt_000001",
  "video_id": "intrusion_climb-over-fence_rgb_0007_cctv3",
  "request_id": "intrusion_climb-over-fence_rgb_0007_cctv3_f000016_000026_000036",
  "frame_indices": [16, 26, 36],
  "frame_times_sec": [0.533, 0.867, 1.2],
  "risk_state": "danger",
  "stage1_actions": [
    "stand, carry/hold (an object)",
    "stand, carry/hold (an object)",
    "stand, carry/hold (an object)"
  ],
  "snapshot_images": [
    "...t000016.jpg",
    "...t000026.jpg",
    "...t000036.jpg"
  ],
  "overlay_images": [
```



```
"...F1_danger.jpg",
"...F2_danger.jpg",
"...F3_danger.jpg"
]
}
```

12. Overlay 이미지

Stage 1 이미지에는 bbox가 그려져 있다.

Stage 2 VLM이 `unsafe` 또는 `danger`를 출력하면, 최종 후처리 단계에서 이미지 상단에 위험 상태 badge를 추가한다.

```
UNSAFE
DANGER
```

저장 위치:

```
benchmark2/runs/{run_id}/final/overlay_images/
```

현재는 위험 request 하나당 이미지 3장에 overlay를 생성한다.

예를 들어 danger request가 5개라면:

```
5 requests × 3 frames = 15 overlay images
```

가 생성된다.

13. 비디오 출력

YOWOV3에는 bbox 이미지를 mp4로 묶는 기능이 있다.

옵션:

```
--stage1-make-video
```

현재 기본값은 꺼져 있다.

현재 목적은 통합 파이프라인 동작 검증이므로, 영상 생성보다 로그와 이미지 결과를 우선한다. 필요하다면 발표용 결과 생성 단계에서 옵션을 켤 수 있다.

14. 특정 영상만 실행하는 방법

전체 데이터를 모두 실행하면 시간이 오래 걸리므로, 특정 영상 하나만 선택하여 빠르게 테스트할 수 있다.

```
--frame-video-filter intrusion_climb-over-fence_rgb_0007_cctv3
--max-frame-videos 1
```

이 옵션을 사용하면 전체 데이터가 아니라 지정한 영상 하나만 Stage 1에 입력된다.

15. 실행 예시

```
cd /home/capstone2/zroact-stage2

python3 pipeline/main.py
--frames-dir /home/capstone2/zroact-stage2/benchmark2/data2/no_bbox_frames
--frame-video-filter intrusion_climb-over-fence_rgb_0007_cctv3
--max-frame-videos 1
--run-id data2_one_video_stage2_only
--prompt benchmark2/prompts/no_action_timev1.txt
--stage1-pretrain-path /home/capstone2/zroact-stage1/ckpt/Y0W0v3/checkpoint/
M23/ema_epoch_9.pth
--stage1-sample-rate 10
--vlm-limit 5
```

16. 처리 시간 측정 결과

테스트 조건

테스트 영상:
intrusion_climb-over-fence_rgb_0007_cctv3

영상 길이:
400 frames / 30 FPS $\approx$ 13.3초

Stage 1 sample rate:
10

Stage 2 VLM request 제한:
5

단계별 측정 결과

처리 단계	시간
선택 프레임 준비	0.0165초
Stage 1 YOWOv3	13.8044초
Stage 2 manifest 생성	0.0034초
Stage 2 VLM	16.4192초
최종 로그 및 overlay 생성	0.4233초
전체 처리 시간	30.6685초

VLM 순수 생성 시간

총 VLM generation latency: 7.8754초
평균 latency: 1.5751초/request

Stage 2 전체 시간이 더 긴 이유는 Qwen 모델 로딩 시간이 포함되어 있기 때문이다.

17. 현재 구조의 한계

현재 통합 파이프라인은 순차 실행 구조이다.

Stage 1 전체 완료
→ Stage 2 VLM 실행
→ 최종 로그 및 overlay 생성

따라서 전체 처리 시간은 다음과 같이 합산된다.

전체 시간
= Stage 1 처리 시간
+ Stage 2 모델 로딩 및 추론 시간
+ 후처리 시간

현재 주요 병목은 다음 두 가지이다.

1. Stage 1 YOWOv3 처리 시간
2. Stage 2 Qwen 모델 로딩 및 VLM 추론 시간

18. 병렬화 이후 개선 가능성

현재 `sample_rate=10` 기준 측정값은 다음과 같다.

Stage 1: 약 13.8초
Stage 2 VLM: 약 16.4초
전체: 약 30.7초

Stage 1과 Stage 2를 병렬화하면, 이론적으로 전체 시간은 단순 합산이 아니라 두 처리 시간 중 큰 값에 가까워질 수 있다.

예상 처리 시간
 $\approx \max(\text{Stage 1}, \text{Stage 2}) + \text{추가 overhead}$
 $\approx \text{약 } 17\sim 20\text{초}$

또한 Qwen VLM을 worker 형태로 미리 로딩해두면 모델 로딩 시간을 줄일 수 있다.

Stage 1: 약 13.8초
VLM 순수 추론: 약 7.9초

이 경우 전체 시간은 이론적으로 약 14~16초 수준까지 줄어들 가능성이 있다.

단, 이 수치는 실제 병렬 구조를 구현하고 측정한 결과가 아니라 현재 단계별 처리 시간을 기준으로 계산한 예상치이다.

19. 현재 구현 완료 항목

- 입력 영상 또는 프레임 폴더 처리
 - 특정 영상 하나 선택 실행
 - Stage 1 YOWOv3 자동 실행
 - Stage 2 manifest 자동 생성
 - Qwen3.5 VLM 자동 실행
 - normal / unsafe / danger 상태 로그 생성
 - unsafe / danger 위험 이벤트 로그 생성
 - 위험 상태 overlay 이미지 생성
 - 단계별 실행 시간 측정
 - 기존 benchmark2 실험 구조 유지
-

20. 추후 구현 항목

- Stage 1과 Stage 2 병렬 처리
- Qwen VLM worker 상시 로딩
- 실시간 CCTV stream 처리
- 프론트엔드 및 백엔드 API 연결
- 위험 이벤트 영상 자동 생성
- HTML 기반 시뮬레이션 화면 또는 viewer 자동 생성
- Human GT coverage 기반 평가 자동화

21. 발표 또는 시연 방향

발표 현장에서 Stage 1부터 Stage 2까지 모든 모델을 실시간으로 실행하면 GPU 환경, 모델 로딩 시간, 오류 가능성 때문에 발표 흐름이 끊길 수 있다.

따라서 초기 시연은 실제 통합 파이프라인의 결과를 기반으로 한 replay 방식이 적절하다.

CCTV 영상 재생
→ Stage 1 bbox 및 action label 표시
→ 10프레임 간격 이미지 3장 선택
→ Stage 2 VLM 입력 표시
→ normal / unsafe / danger 결과 표시
→ 위험 로그 누적

이 방식은 단순한 결과 캡처가 아니라, 실제 실행된 파이프라인의 처리 흐름을 시간 순서대로 보여주는 시뮬레이션이다.

22. 최종 요약

현재 구현한 ZroAct 통합 프레임워크는 기존 Stage 1 YOWOV3와 Stage 2 Qwen3.5 VLM 구조를 유지하면서, 입력 영상 또는 프레임 폴더를 받아 행동 탐지, Stage 2 입력 sequence 생성, 침입 위험 상태 판단, 최종 위험 로그 및 overlay 이미지 생성까지 한 번에 수행한다.

현재 버전은 실시간 서버가 아니라 오프라인 통합 실행 구조이다.

향후에는 Stage 1과 Stage 2 병렬화, Qwen worker 상시 로딩, HTML 기반 replay 시뮬레이션을 추가하여 발표용 시연과 실제 운영 구조에 가까운 형태로 확장할 예정이다.